
P.073 - Organization Of Design Data

Release 015

Embention Sistemas Inteligentes, S.A.

2024-11-18

Contents

- P.073 - Organization Of Design Data 3
 - 1. Purpose and Scope..... 3
 - 2. Description of activities..... 3
 - 2.1. Choose repository 3
 - 2.2. Choose item..... 4
 - 2.3. Choose position in the directory tree 9
 - 2.4. Choose data 11
 - 2.5. Exception 12
- Example DFS2_CreateImage 14
 - Procedure 14

P.073 - Organization Of Design Data

1. Purpose and Scope

This document describes where to store the design data (Design Data in ARP/DOs) that are generated in the company every day. For example: source code, printed circuit boards, 3D drawings, test cases, documents, etc. All this data will be organized in **repositories** owned by Embention in your space within [GitHub](#).

There is an [exception](#) for the use of this procedure applicable to software repositories for the development of internal tools.

2. Description of activities

The procedure to be followed consists of 4 decisions to be taken:

2.1. Choose repository

First of all, select the repository (hereinafter, repo) where the Design Data we are working on will be stored. The choice of repo for this Design Data must be made taking into account its relations with another Design Data. If the Design Data is strongly dependent on another Design Data that is already in a repo, that will be the repo we should use, regardless of departments, repo size, etc. The reason for doing this is to **avoid inconsistencies** between them since they are related. For example, as you know Veronte and Veronte App are closely related. A modification in Veronte usually requires a modification in Veronte App. For this reason, we will place them in the same repo and our procedures will help us to avoid inconsistencies between them.

All the design work is done by starting an Issue from a repo. In general, the repo where we will store the data will be the same one where the Issue was created, although this might not be the case. In some cases an Issue is created in the wrong repo due to lack of knowledge of the repos or because it responds to a problem that is thought to be there. If this is the case, both the Issue and the Design Data should be moved to the correct repo as soon as possible. That

the Issues and Design Data are in the right place is something that should be checked in the Pull-Request, before the move to develop.

Although more types may exist in the company, for configuration management, etc., in relation to the Design Data there will be only 3 types of repo:

1. PRODUCT repo

It is a repo with access from the different departments, and that contains all the Design Data related to that product. As soon as a product reuses data from another product (and not before), this data will be moved to another repo (see type 3) and will be used as a sub-module from the product repos.

2. PROJECT repo

It is a repo with access from the departments involved in the project, and that contains all the Design Data related to that project and **exclusive** to it. For example, if as part of a project a new Veronte functionality is developed, in general this functionality will be developed in the product repo. The project may also (only if necessary) use as submodules the repos of the products used in it.

3. LIBRARY repo

This is a repo with normally exclusive access from a department. The data of these repos are reused in the different products/projects. Something very important: these repos are used as submodules, and therefore they are visible from the repos that use them, but not the other way around.

2.2. Choose item

Once we have selected the repo, the second thing to do is to identify the item in which the Design Data we are working on will be stored and its parent item. In order to be able to find the information easily and even eventually take the information to a repo library, a uniform criterion is going to be adopted in the company inspired by the ARP4754A. The item and its folder structure will depend on the type of Design Data we want to store, which could be:

1. item of type SYSTEM or type PROJECT

The name of the root directory of the item must identify it univocally and be composed only of alphanumeric characters (without accents) and

underscores **SW_**. (i.e. Veronte, CEX, SUC, etc.) The content of these items shall be organized using the required directories from the following table:

 **Note**

A system item may itself contain a system item.

Directory	Contents
artifacts	system requirements, designs, system test cases, etc.
docs	manuals, procedures, references, generation scripts, etc.
items	subitems, an item can contain other items of any type
sys	description and information about the system and its associated releases

2. item of type SOFTWARE

These items shall contain software design data. The name of SOFTWARE type directories shall consist of the prefix **SW_** followed by the unique identifier of the item. (i.e. sw_1xVeronte, sw_CEX, sw_SUC, etc.) The content of these items shall be organized using the required directories from the following table:

Directory	Contents
artifacts	software requirements, software design, test cases, etc.
docs	manuals, procedures, references, generation scripts, etc.
items	subitems, all of them will be of software type (software modules)
code	application: source code, includes, tests, projects, etc.

3. item of type HARDWARE

These items shall contain mechanical, hardware, wiring or production design data. The name of HARDWARE type directories shall consist of the prefix **3d_** (mechanical), **hw_** (hardware), **har_** (wiring) or **pro_**

(production) followed by the unique identifier of the item. (i.e. 3d_Veronte, har_JTAG, hw_SUC, etc.). The content of these items shall be organized using the required directories from the following table:

Directory	Contents
artifacts	hardware requirements, test cases, etc.
docs	manuals, procedures, references, generation scripts, etc.
items	subitems, all of them will be hardware type (hardware modules)
design	hardware design: schematic, PCB, tests, simulations, etc.
3d	3D designs of the item, sketches, drawings, etc.
pro	consumables, etc.

4. item of type PDI

These items shall contain design data from device or software configuration files. The name of the PDI type directories shall consist of the prefix **pdi_** followed by the unique identifier of the item. (i.e. pdi_M400, pdi_CEX_B, sim_M600, etc.). For repositories of type PRODUCT, hardware versions and hardware variants will be considered variants of the item pdi (e.g. in the 1xVeronte repository each hardware version is considered a variant, as is the BCS). The content of these items shall be organized using the required directories from the following table:

Directory	Contents
artifacts	configuration requirements, test cases, etc.
docs	incoming and outgoing documents, datasheets, manuals, etc.
items	subitems, all of them will be of type pdi
pdi	configuration files (pdi files)
sim	simulator models and simulation presets

Directory	Contents
gui	user interface configurations (Veronte Ops, etc.)
variants	pdi type variants (1XV45, 1XV48, etc.)

5. item of type CONTAINER

This typology will be used when the number of items is ≥ 4 . The content of these items will be organized using the required directories from the table below.

Container items can be classified in folders. Only the type of item is determined at the last level, with the prefix in the folder name when it reaches a name of relevant information. Relevant information is considered to be the name of the manufacturer and the standard (ISO, DIN UNE...). Two examples are shown below: The use of **items** of container type for development items shall be avoided.

Directory	Contents
item 0	any kind of item
item 1	another item of the same type as the item 0
...	
item n	another item of the same type as the item 0

6. item of type FPGA

These items will contain hardware description and design data. The name of the FPGA type directories will consist of the prefix **fpga_** followed by the unique identifier of the item. (i.e. fpga_1xVeronte, fpga_CEX, fpga_SUC, etc.) The content of these items shall be organized using the required directories from the following table:

Directory	Contents
artifacts	hardware description requirements, hardware design, etc.

Directory	Contents
docs	manuals, procedures, references, generation scripts, etc.
items	subitems, all of which will be fpga type (fpga modules)
code	application: hardware description code, projects, etc.

In addition to the folders listed above, there may be folders:

- **github:** located in the root of the repo, contains the necessary files for the configuration of Issues and pull-requests templates.
- **CI:** located in the root of any CI integrated item directory, contains the files needed for task automation from CI.
- **variants:** located at the root of any item directory whose design contains different variants. This folder will contain one directory per variant, whose name will be that of its 5-digit identifier. The structure of the variant directories will be the same as their root directory, containing only files specific to the corresponding variant, all files that apply to all variants must be stored in the corresponding place in the root directory.

 Important**Item nomenclature convention**

1. The names of items shall always follow the structure corresponding to the type of item (**prefix + identifier**), except in the case of system items that have no prefix.
2. The identifier must be distinctive of the item, trying to synthesize as much as possible. Only alphanumeric characters (without accents) and underscores may be used (i.e. GimbalPro, hw_VeronteGimbal, sw_VeronteGimbalStudio).
3. For each system item we will use the same identifier, so that the distinction between design content is the prefix (i.e. sw_CEX, hw_CEX, har_CEX)
4. The concatenation **prefix + identifier** must be unique in the repo and among all the repos of the company.
5. For the **sw** , **3d** , **hw** and **har** main items of the system/project corresponding to the repo, the identifier of the item shall correspond to the repo name. If the repo name contains hyphens **-** they will be replaced by underscore **_** . (i.e. hw_1xVeronte [repo 1xVeronte], sw_Veronte_SUC [repo Veronte-SUC])

2.3. Choose position in the directory tree

Once we know the repo, the parent item and item we are creating, we have to locate that folder in the directory tree of the repo. The following will be taken into account:

- In order to avoid long paths, the item (system, hardware, software or container) will be placed attending to a **usage criterion**, that is, it will go to the highest possible level of a repo's tree if no other item in the repo uses it.
- The usage criterion establishes a one-way dependency, i.e., the item it uses **sees** the one used but not the other way around.
- Folders that are on the same level can also be used and **seen**.

-
- An exception is allowed for better organization: **tests** and information that is unique to an item can be placed as a subitem of it. This exception is the only one that allows upward visibility.
 - Since hardware items do not use the software, but the system does, software and hardware items in a system will be at the **same level** in the hierarchy, as is the case for items according to ARP4754A.
 - If an item is used by 2 or more items in different repos, the item in question will be separated to a library repo and will be included in the hierarchy of each of the other items according to the same rules, but as a **submodule** (submodules will be distinguished in the item hierarchy by the prefix  in front of the name).
 - The **submodule** will be placed at the same level as the item that needs it.
 - If an item is used by 2 or more items in the same repo, for simplicity, it can be placed at the same level as these if they are. If the hierarchy prevents it, it will be done as in the previous case, with submodules.
 - There may be **items** of different types at the same level. Also applicable to **submodules**.
 - The criterion of use implies that a complex system generates **recursively** a complex tree. For this reason, relative paths will be used to include files and if this is not possible and absolute paths cannot be handled, the case will be studied separately.



Example of directory organization

2.4. Choose data

Once all the above is understood, we can save our Design Data. The following should be taken into account:

- The data to be saved will depend in each case on the type of item.
- All the required information that makes our Design Data to be **completely defined** and unique must be saved.
- The files that store this information will be subject to version control, i.e., they will change as revisions and improvements are made. This applies to text files as well as binary files.
- The history of these files in the repo is enough to identify versions so **no other files** will be created for further development (e.g. board.sch -> board v2-definitive.sch).
- In order to avoid inconsistencies, **dependent** files, i.e. those resulting from other files when performing a non-design operation (e.g. a compiled file, an executable, a screenshot, a Gerber, etc.) should not be stored in the folder structure.

- Dependent files will be stored in the **Release** section of the corresponding repo.

2.5. Exception

In order to simplify the development and deployment of the internal tools developed in the company, a more relaxed structure is proposed, adapted to the needs of each tool. The main departments whose repositories will be able to apply this exception are Odoo, Tools, CI and IT.

The conditions are as follows:

- It applies to an entire software repository.
- You cannot include an **exception** item within a repository that already meets the Design Data.
- Contains software and/or tool configuration for internal use **not certifiable**.

Structure:

- The main items should go at the root of the repository.
- In general, the simplest possible structure should be sought, ensuring order within the repository, avoiding redundant folder names such as **code** within a repository that is exclusively for code.

Example of a repository structure of the CI department, of software type **sw_CI_Libs** whose three main items are in the root and each one contains a simple and little nested structure, which allows to clearly identify the content.

```
sw_CI_Libs
├─ README.md
├─ .github
│   └─ workflows
│       └─ ... workflows
│   └─ actions
│       └─ ... actions
├─ github
│   └─ workflows
│       └─ ... workflows
├─ jenkins
│   └─ ci_jobs
│       └─ ... jobs
│   └─ libraries
│       └─ python
│           └─ ... python library
│       └─ groovy
│           └─ ... groovy library
└─ libraries
    └─ ... external libraries for external use
```

Example DFS2_CreatelImage

Where to save the source code and its associated elements (artifacts, docs, ...) of a PC application development (e.g. DFS2_CreatelImage) that generates an image with the DFS2 file system to be saved on a MicroSD?

Procedure

1. Choose repository

As MicroSDs with DFS2 are used in several products (1xVeronte, 4xVeronte, etc.), this application is common to them, so the best place is Vlibs (the common part, which is a LIBRARY repo).

2. Choose element type

It will be a software type element. For example DFS2_CreatelImage (with subfolders artifacts, code, docs, items).

3. Choose position in the directory tree

Following the **usage criteria**, and since Vlibs is a container, DFS2_CreatelImage would go directly to the root (Vlibs/DFS2_CreatelImage). As DFS2 is in Vlibs/DFS2 there would be no problem, being at the same level, DFS2_CreatelImage can see DFS2. However, it can be even better (see exception) if we put it inside the DFS2 library since there is an exclusive relationship, like a test: if DFS2 changes, DFS2_CreatelImage will change, the more together the better. Therefore, we can place it inside the DFS2 items folder. This way, if someday for any reason we had to detach DFS2 to another repo, DFS2_CreatelImage would go with it automatically.

4. Choose data

Inside the DFS2_CreatelImage/Code folder we will find all the application code. In this case as the application is in python, we will place all the python scripts needed to run the application. A Readme.md is also included, with information on how to run the application.